

PE GUÍA DE ESTRUCTURA DEL PROYECTO — GRUPO 02

Sistema Web de Clima y Datos Meteorológicos del Perú · Arquitectura por Capas · 2026

1. Archivos en la Raíz del Proyecto (Configuración Global)

CONFIG `.env.example`: Plantilla de variables de entorno (URL de base de datos, TTL de caché, URLs de APIs externas y CORS).

CONFIG `vercel.json`: Configuración de enrutamiento y despliegue serverless en la nube (Vercel) para frontend y backend.

BACKEND `requirements.txt`: Declaración de librerías Python requeridas (FastAPI, Uvicorn, SQLAlchemy, Pydantic, HTTPX).

SERVERLESS `api/index.py`: Punto de entrada (*handler serverless*) para desplegar la API FastAPI en Vercel.

DOCS `README.md`: Manual técnico del repositorio con descripción, catálogo de endpoints e instrucciones de ejecución.

2. Estructura del Backend (FastAPI + Python)

📁 backend/app/ (Núcleo del Servidor)

CORE `main.py`: Inicializador de FastAPI. Configura CORS, eventos de inicio (*startup* para crear tablas y sembrar datos) e incluye los routers.

CONFIG `config.py`: Carga las variables de entorno usando Pydantic Settings para configuración centralizada y segura.

DATABASE `database.py`: Crea el motor SQLAlchemy (`create_engine`), sesión y generador de dependencias `get_db()`.

📁 backend/app/models/ (Modelos de Base de Datos - Tablas ORM)

TABLA `peru_geo.py`: Define las tablas `departments` (25 dptos) y `cities` (40+ ciudades con altitud y coordenadas). Relación 1 a N.

TABLA `weather_cache.py`: Define la tabla `weather_cache` para almacenar respuestas JSON y otorgar resiliencia offline.

TABLA `favorite.py`: Define la tabla `favorite_cities` para marcar y persistir ciudades favoritas.

📁 backend/app/routers/ (Controladores / Endpoints REST)

`weather.py`: Endpoints `/forecast`, `/current` y `/overview` (resumen nacional de 25 departamentos).

`cities.py`: Endpoint `/cities` (búsqueda y filtrado por región Costa, Sierra, Selva).

`departments.py`: Endpoint `/departments` (lista de departamentos y capitales).

`alerts.py`: Endpoint `/alerts` (alertas calculadas por umbrales meteorológicos).

`compare.py`: Endpoint `/compare` (comparador de 2 a 4 ciudades simultáneas).

history.py: Endpoint /history (historial de 7, 15 o 30 días y estadísticas).

rankings.py: Endpoint /rankings (Top 5 ciudades más calurosas, frías, lluviosas y ventosas).

export.py: Endpoint /export/csv (generación y descarga de reportes en archivo CSV).

favorites.py: Endpoint /favorites (guardar y listar ciudades favoritas).

📁 backend/app/services/ y seed/ (Lógica de Negocio y Datos)

LOGIC `services/weather_service.py`: Conexión con Open-Meteo vía HTTPX, lógica de caché en RAM, normalización de códigos WMO al español y cálculo de tendencias.

LOGIC `services/alert_engine.py`: Motor de reglas físicas para emitir alertas ($UV \geq 11$, heladas $\leq 0^{\circ}\text{C}$, vientos fuertes).

SEED `seed/seed_peru.py`: Sembrador de datos iniciales con la geografía completa de los 25 departamentos y ciudades peruanas.

TESTS `backend/tests/`: Suite de pruebas automatizadas con **Pytest** para verificar el correcto funcionamiento de los endpoints.

3. Estructura del Frontend (React 19 + TypeScript + Vite)

📁 frontend/ (Archivos de Configuración del Cliente)

CONFIG `package.json`: Define las librerías frontend (React 19, Leaflet, Recharts, Lucide, Tailwind CSS) y scripts de ejecución.

CONFIG `vite.config.ts`: Configuración de Vite para compilación ultrarrápida y proxy de desarrollo.

CONFIG `tailwind.config.js` / `postcss.config.js`: Sistema de diseño, paleta de colores y clases utilitarias de Tailwind.

📁 frontend/src/components/ (Componentes Visuales de la Interfaz)

`CurrentWeatherHero.tsx`: Tarjeta principal del clima actual (temperatura, hora oficial UTC-5, amanecer/atardecer).

`KeyMetricsGrid.tsx`: Las 6 tarjetas métricas clave (Índice UV, Humedad, Viento, Presión, Precipitación, Nubosidad).

`HourlyForecast.tsx`: Carrusel horizontal con el pronóstico de las próximas 24 horas.

`DailyForecast.tsx`: Pronóstico extendido a 7 días con barras proporcionales de rango térmico (mín-máx).

`WeatherCharts.tsx`: Gráficos interactivos construidos con **Recharts** (temperatura, lluvia, viento).

`WeatherMap.tsx`: Mapa interactivo del Perú desarrollado en **Leaflet** con capas térmicas y de lluvia.

`CompareSection.tsx`: Módulo de comparación lado a lado de 2 a 4 ciudades peruanas.

`HistorySection.tsx`: Vista de tendencias climáticas históricas y botón de exportación CSV.

`AlertsSection.tsx`: Panel de notificaciones y alertas de protección civil.

`RankingsSection.tsx`: Tablas de posiciones con los extremos climáticos nacionales.

`SkeletonLoader.tsx`: Animaciones de carga (*skeletons*) para una experiencia de usuario fluida mientras se consultan los datos.

📁 frontend/src/services/ y types/ (Comunicaciones y Tipos)

API **services/api.ts**: Módulo cliente con llamadas `fetch()` tipadas hacia los endpoints del backend.

TYPES **types/weather.ts**: Definición de interfaces TypeScript para modelos de datos (Ciudad, Departamento, Pronóstico, Alertas).

ENTRY **App.tsx & main.tsx**: Ensamblado principal de la aplicación, control de estado global y renderizado en el DOM.